

Document Number: P1668R0

Date: 2019-06-10

Audience: Evolution Working Group (EWG), Evolution Working Group Incubator (SG17)

Reply-to: Erich Keane <erich.keane@intel.com>

Enabling constexpr Intrinsics By Permitting Unevaluated inline-assembly in constexpr Functions

Revision History:

R0: Initial Version.

Introduction and Motivation:

This paper proposes altering the rules of constexpr functions to permit its definition to contain asm-definitions in cases where it is not evaluated at compile-time. This is particularly useful when attempting to make certain processor intrinsic functions constexpr. While there are currently techniques to make these functions constexpr, such as implementing them as compiler builtins, these strategies are only possible with compiler support. Additionally, handwritten assembly versions of functions are often present in user code where compiler support isn't possible. For example, consider a simple Fused Multiply/Add (FMA) implementation:

```
double fma(double b, double c, double d) {  
    asm("vfmadd132sd %0 %1 %2"  
        : "+x"(b)  
        : "x" (c), "x" (d)  
        );  
    return b;  
}
```

In some codebases, a function like this may be used quite commonly with the intent of optimizing certain algorithms. However, there are three massive inconveniences to this implementation; unless explicitly documented it isn't clear what it does to someone who doesn't know what 'fma' means, it cannot be constant folded by a compiler, and it cannot be used in a constexpr or consteval context.

However, if this proposal is accepted an implementer could use `std::is_constant_evaluated()` to make this function constexpr and solve both of those issues. Consider the following:

```
constexpr double fma(double b, double c, double d) {  
    if (std::is_constant_evaluated())  
        return b * c + d;  
    asm("vfmadd132sd %0 %1 %2"  
        : "+x"(b)  
        : "x" (c), "x" (d)  
        );  
    return b;  
}
```

It is now completely clear what this function does, since there is a non-assembly version. It can also be used in a constexpr context resulting in a significant performance improvement. Finally, the runtime performance of the inline assembly version isn't sacrificed in order to make this possible. This function is admittedly quite simple (and in fact, the GNU compiler will actually take the non assembly version and turn it into the equivalent of the assembly directive), however many more intrinsics exist that are not quite so simple to get the compiler to optimize. Additionally, user written assembly constructs are typically done when the user notices the compiler fails to produce optimal assembly.

Wording (vs [N4810]):

Change in [expr.const] /4

An expression **e** is a *core constant expression* unless the evaluation of **e**, following the rules of the abstract machine (6.8.1), would evaluate one of the following expressions:

- **this** (7.5.2), except in a constexpr function or a constexpr constructor that is being evaluated as part of **e**;
- ...
- a *throw-expression* (7.6.18) or a dynamic cast (7.6.1.6) or **typeid** (7.6.1.7) expression that would throw an exception; ~~or~~
- **an asm-definition; or**
- an invocation of the **va_arg** macro (17.13.1).

Change in [decl.constexpr]/3

The definition of a constexpr function shall satisfy the following requirements:

- its return type shall be a literal type;
- each of its parameter types shall be a literal type;
- it shall not be a coroutine (9.4.4);
- its *function-body* shall not enclose (Clause 8)
 - ~~— an asm-definition,~~
 - a **goto** statement,
 - an identifier label (8.1),
 - a definition of a variable of non-literal type or of static or thread storage duration or for which no initialization is performed.

References:

[N4810] "Working Draft, Standard for Programming Language C++", <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4810.pdf>, 2019